



# THE AI ENGINEER'S TOOLKIT

# Autonomous AI Agents

---

*A Hands-On Guide to  
LangChain & LangGraph*

---

**KAIRAT YUSSUPOV**

First Edition, June 2026

\newpage

## L I C E N S E

---

---

Copyright © 2026 Kairat Yussupov

Permission is hereby granted, free of charge, to any person obtaining a copy of this book and associated documentation files (the "Work"), to deal in the Work without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Work, and to permit persons to whom the Work is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Work.

THE WORK IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE WORK OR THE USE OR OTHER DEALINGS IN THE WORK.

---

**Edition 1.0**

**Published** June 2026

**License** MIT

**Series** The AI Engineer's Toolkit

\newpage

# Table of Contents

---

## **Introduction**

### **Part I: Foundations**

- Chapter 1: Understanding LLMs and the Need for Orchestration
- Chapter 2: Getting Started with LangChain
- Chapter 3: Core LangChain Components

### **Part II: Building with LangChain**

- Chapter 4: Prompt Engineering with Templates
- Chapter 5: Chains – Composing LLM Calls
- Chapter 6: Memory – Giving Context to Conversations
- Chapter 7: RAG – Connecting LLMs to Your Data
- Chapter 8: Tools and Toolkits

### **Part III: Agents with LangGraph**

- Chapter 9: From Linear Chains to Graphs
- Chapter 10: StateGraph – The Heart of LangGraph
- Chapter 11: Nodes, Edges, and Conditional Routing
- Chapter 12: Agent Loops and Reasoning
- Chapter 13: Human-in-the-Loop Patterns
- Chapter 14: Persistence and Long-Term Memory

### **Part IV: Production and Beyond**

- Chapter 15: Observability with LangSmith
- Chapter 16: Deploying Agentic Applications
- Chapter 17: Multi-Agent Systems
- Chapter 18: From Prototype to Production



## **Appendices**

- A: Environment Setup
- B: Python Virtual Environments
- C: Common Troubleshooting
- D: Further Resources

\newpage

# Introduction

---

We are living through a fundamental shift in software development. Large Language Models (LLMs) have evolved from novelty chatbots into capable reasoning engines that can write code, analyze documents, control browsers, and take actions in the real world. But an LLM alone is not an application.

The challenge is orchestration. How do you connect an LLM to your data? How do you give it tools? How do you manage conversation history, retry logic, and multi-step reasoning? How do you build something that works reliably in production?

This book answers those questions using two of the most popular open-source frameworks in the AI ecosystem: **LangChain** and **LangGraph**.

LangChain provides the building blocks — prompt templates, model abstractions, memory, retrieval, and tool integrations. LangGraph takes those building blocks and arranges them into dynamic, stateful graphs that can loop, branch, and self-correct. Together, they form a complete toolkit for building autonomous AI agents.

## What You Will Learn

- How LLMs work and why they need orchestration frameworks
- LangChain fundamentals: prompts, chains, memory, RAG, and tools
- LangGraph fundamentals: state graphs, nodes, edges, and conditional routing
- How to build agents that reason, use tools, and remember context
- Production patterns: observability, deployment, and multi-agent systems
- How to combine LangChain and LangGraph for real-world applications

## Who This Book Is For

This book is for software developers, data scientists, and technical builders who want to build AI-powered applications. You should know basic Python. No prior experience with LLMs or LangChain is required.

## How to Use This Book

Each chapter builds on the previous one. If you are entirely new, read sequentially. If you have some experience with LangChain, feel free to jump to Part III on LangGraph. Every code example is self-contained and can be run with a minimal setup (see Appendix A).

All code examples use Python 3.11+ and the latest versions of LangChain and LangGraph as of mid-2026.

---

# Part I: Foundations

---

## Chapter 1: Understanding LLMs and the Need for Orchestration

### 1.1 What Are Large Language Models?

A Large Language Model is a neural network trained on vast amounts of text data to predict the next token (roughly, a word or sub-word) given a sequence of previous tokens. Through this simple objective, trained at massive scale, these models develop remarkable capabilities: they can answer questions, summarize documents, write code, translate languages, and reason about complex problems.

At their core, LLMs are:

- **Stateless** — Each call is independent. The model does not remember previous interactions.
- **Probabilistic** — Outputs are sampled, not computed deterministically. The same prompt can yield different responses.
- **Knowledge-limited** — Their training data has a cutoff date. They do not know about recent events unless given context.
- **Tool-less** — They can only produce text. They cannot query databases, call APIs, or browse the web on their own.

These limitations are not bugs; they are architectural constraints. The entire LangChain/LangGraph ecosystem exists to address them.

## 1.2 What Is Orchestration?

Orchestration is the coordination of multiple LLM calls, data sources, and external tools into a coherent workflow. Consider a simple Q&A bot:

1. User asks a question.
2. Search the knowledge base for relevant documents.
3. Insert those documents into a prompt as context.
4. Call the LLM with the enriched prompt.
5. Return the answer.
6. Store the conversation in memory for follow-up questions.

Each step is simple. Orchestrating them reliably is the hard part. That is where LangChain comes in.

## 1.3 Why LangChain and LangGraph?

LangChain (launched in late 2022) quickly became the standard framework for LLM application development. Its strengths are:

- **Model agnosticism** — The same code works with OpenAI, Anthropic, Google, open-source models, and hundreds of others.
- **Composable primitives** — Prompts, models, parsers, and retrievers snap together like Lego bricks.

- **Rich ecosystem** — Hundreds of integrations for vector stores, document loaders, embedding models, and more.

LangGraph (launched in early 2024) extends LangChain into the realm of autonomous agents:

- **Stateful graphs** — Define workflows as graphs with persistent state across steps.
- **Loops and branching** — Agents can retry, reconsider, and take different paths based on intermediate results.
- **Human-in-the-loop** — Pause execution and wait for human approval at critical decision points.
- **Durable execution** — Survive process restarts through checkpointing.

Together, they give you everything you need to build from a simple chatbot to a sophisticated multi-agent system.

---

## Chapter 2: Getting Started with LangChain

### 2.1 Installation

Create a Python virtual environment and install LangChain:

```
python -m venv .venv
source .venv/bin/activate
pip install langchain langchain-openai python-dotenv
```

Set your API keys in a `.env` file:

```
OPENAI_API_KEY=sk-...
```

Load them in your code:

```
from dotenv import load_dotenv
load_dotenv()
```

## 2.2 Your First LangChain Application

Let us call an LLM directly through LangChain:

```
from langchain_openai import ChatOpenAI
from langchain_core.messages import HumanMessage

llm = ChatOpenAI(model="gpt-4o-mini")

response = llm.invoke([HumanMessage(content="Say
hello in French.")])
print(response.content)
```

Output:

```
Bonjour !
```

## 2.3 The Core Interface

LangChain wraps all LLMs behind a consistent interface. Whether you use OpenAI, Anthropic, Google, or a local model via Ollama, the calling pattern is the same:



```
llm.invoke(messages)      # single call
llm.batch(messages_list)  # multiple calls
llm.stream(messages)      # streaming
```

Messages are typed objects:

```
from langchain_core.messages import SystemMessage,
HumanMessage, AIMessage

messages = [
    SystemMessage(content="You are a helpful as-
sistant."),
    HumanMessage(content="What is the capital of
Japan?"),
    AIMessage(content="Tokyo."),
    HumanMessage(content="What is its
population?"),
]
```

The `SystemMessage` sets the behavior, `HumanMessage` represents user input, and `AIMessage` represents model responses.

## 2.4 Temperature and Model Parameters

LLM parameters control the variability and creativity of responses:

```
llm = ChatOpenAI(
    model="gpt-4o-mini",
    temperature=0.7,    # 0.0 = deterministic, 1.0
                        # = creative
    max_tokens=1024,    # maximum response length
    timeout=30,         # request timeout in
                        # seconds
)
```

For factual tasks, use low temperature (0.0–0.2). For creative tasks, use higher values (0.7–1.0).

## 2.5 Streaming

For a better user experience, stream tokens as they are generated:

```
for chunk in llm.stream([HumanMessage(content="Tell
me a short story.")]):
    print(chunk.content, end="", flush=True)
```

Each chunk is an `AIMessageChunk` containing a small piece of the response.

---

## Chapter 3: Core LangChain Components

LangChain is organized around a handful of core abstractions. Understanding them is key to using the framework effectively.

### 3.1 Chat Models

Chat models accept a list of messages and return a message. They are the primary interface for interacting with LLMs. LangChain supports:

- `ChatOpenAI` — OpenAI models
- `ChatAnthropic` — Anthropic Claude models
- `ChatGoogleGenerativeAI` — Google Gemini models
- `ChatOllama` — Local models via Ollama

All implement `BaseChatModel` with the same `invoke`, `batch`, and `stream` interface.

### 3.2 Prompt Templates

Prompt templates sanitize and structure inputs before sending them to the model:

```
from langchain_core.prompts import ChatPromptTemplate

template = ChatPromptTemplate.from_messages([
    ("system", "You are an expert in {topic}.",),
    ("human", "Answer this question: {question}"),
])

prompt = template.invoke({
    "topic": "astrophysics",
    "question": "What is dark matter?",
})
```

The `invoke` method fills in the placeholders and returns a list of messages ready for the model.

### 3.3 Output Parsers

Output parsers transform raw model output into structured data:

```
from langchain_core.output_parsers import StrOutputParser, PydanticOutputParser
from pydantic import BaseModel, Field

class Recipe(BaseModel):
    name: str = Field(description="Recipe name")
    ingredients: list[str] = Field(
        description="List of ingredients"
    )
    steps: list[str] = Field(description="Cooking steps")

parser = PydanticOutputParser(pydantic_object=Recipe)

template = ChatPromptTemplate.from_messages([
    ("human", "Create a recipe for {dish}.\n{format_instructions}"),
])

prompt = template.invoke({
    "dish": "spaghetti carbonara",
    "format_instructions": parser.get_format_instructions(),
})

response = llm.invoke(prompt)
recipe = parser.invoke(response)
print(recipe.name)
print(recipe.ingredients)
```

### 3.4 The LCEL (LangChain Expression Language)

LCEL is a declarative syntax for composing components into chains using the `|` operator:

```
chain = template | llm | parser
result = chain.invoke({
    "dish": "spaghetti carbonara",
    "format_instructions": parser.get_format_instructions(),
})
```

Each component in the pipe must accept the output of the previous component. The chain is a `Runnable` that supports `invoke`, `batch`, `stream`, and `astream`.

### 3.5 Runnable Interface

All LangChain components implement the `Runnable` interface:

```
class Runnable:
    def invoke(self, input, config=None) -> Output: ...
    def batch(self, inputs, config=None) -> list[Output]: ...
    def stream(self, input, config=None) -> Iterator[Output]: ...
    def astream(self, input, config=None) -> AsyncIterator[Output]: ...
```

This consistent interface means you can swap components without changing the calling code.

### 3.6 Config and Callbacks

Pass runtime configuration through the `config` parameter:

```
chain.invoke(  
  {"dish": "pizza"},  
  config={  
    "run_name": "recipe-generator",  
    "tags": ["recipe", "italian"],  
    "metadata": {"user_id": "123"},  
  }  
)
```

Callbacks (e.g., for logging) can be attached through config as well.

---

## Part II: Building with LangChain

---

### Chapter 4: Prompt Engineering with Templates

#### 4.1 Why Templates Matter

Hardcoding prompts in string literals is fragile. Templates provide:

- **Separation of concerns** — Prompt logic stays separate from application logic.
- **Reusability** — The same template works with different inputs.
- **Versioning** — Track changes to prompts alongside code.
- **Safety** — Automatic escaping and formatting.

#### 4.2 String Prompt Templates

The simplest template type interpolates variables into a string:

```

from langchain_core.prompts import PromptTemplate

template = PromptTemplate.from_template(
    "Translate the following text into {language}: {text}"
)

prompt = template.invoke({
    "language": "Spanish",
    "text": "Hello, how are you?",
})

```

### 4.3 Chat Prompt Templates

For chat models, use `ChatPromptTemplate` with typed message slots:

```

from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder

template = ChatPromptTemplate.from_messages([
    ("system", "You are a {role} assistant."),
    MessagesPlaceholder(variable_name="history"),
    ("human", "{input}"),
])

prompt = template.invoke({
    "role": "customer support",
    "history": [
        ("human", "My order hasn't arrived."),
        ("ai", "I'm sorry to hear that. Let me check."),
    ],
    "input": "Can you track it?",
})

```



`MessagesPlaceholder` is a slot where message lists are inserted.

## 4.4 Few-Shot Prompting

Provide examples in the prompt to guide model behavior:

```
from langchain_core.prompts import FewShotChatMessagePromptTemplate

examples = [
    {"input": "What is 2+2?", "output": "4"},
    {"input": "What is 5*3?", "output": "15"},
]

example_template = ChatPromptTemplate.from_messages([
    ("human", "{input}"),
    ("ai", "{output}"),
])

few_shot_prompt = FewShotChatMessagePromptTemplate(
    examples=examples,
    example_prompt=example_template,
)

final_prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a math tutor."),
    few_shot_prompt,
    ("human", "{input}"),
])
```

## 4.5 Prompt Composition

Combine multiple templates:

```
instruction_prompt = ChatPromptTemplate.from_template(
    "Answer in {language}."
)
context_prompt = ChatPromptTemplate.from_template(
    "Context: {context}"
)
question_prompt = ChatPromptTemplate.from_template(
    "Question: {question}"
)

combined = instruction_prompt + context_prompt +
question_prompt
```

## 4.6 Best Practices

1. **Be specific** — Vague prompts produce vague responses.
  2. **Provide examples** — Few-shot prompting dramatically improves reliability.
  3. **Use system messages** — Set behavior, tone, and constraints in the system message.
  4. **Limit output scope** — Specify format, length, and structure explicitly.
  5. **Iterate** — Test and refine prompts as you would any code.
-

## Chapter 5: Chains – Composing LLM Calls

### 5.1 What Are Chains?

A chain is a sequence of steps where the output of one step feeds into the next. LangChain provides several built-in chain types and the LCEL for custom compositions.

### 5.2 The Simplest Chain

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

prompt = ChatPromptTemplate.from_template(
    "Tell me a {adjective} joke about {topic}."
)

chain = prompt | llm | StrOutputParser()

result = chain.invoke({"adjective": "funny",
                       "topic": "programmers"})
```

### 5.3 RunnablePassthrough and RunnableParallel

Use `RunnablePassthrough` to pass data through unchanged, and `RunnableParallel` to run branches in parallel:

```

from langchain_core.runnables import Runnable-
Passthrough, RunnableParallel

chain = (
    RunnablePassthrough.assign(
        response=lambda x: llm.invoke(f"Answer:
{x['question']}")
    )
    | (lambda x: f"Question: {x['question']}
\nAnswer: {x['response'].content}")
)

```

`RunnableParallel` runs multiple chains concurrently:

```

summary_chain = prompt1 | llm | StrOutputParser()
keywords_chain = prompt2 | llm | StrOutputParser()

parallel = RunnableParallel(
    summary=summary_chain,
    keywords=keywords_chain,
)

result = parallel.invoke({"text": long_article})

```

## 5.4 Branching and Routing

Route inputs to different chains based on conditions:

```

from langchain_core.runnables import RunnableBranch

technical_chain = prompt_tech | llm_tech | StrOutputParser()
general_chain = prompt_gen | llm_gen | StrOutputParser()

branch = RunnableBranch(
    (lambda x: x["type"] == "technical", technical_chain),
    (lambda x: x["type"] == "general", general_chain),
    general_chain, # default
)

result = branch.invoke({"type": "technical", "question": "What is a binary tree?"})

```

## 5.5 Custom Runnable Functions

For arbitrary logic, wrap functions as runnables:

```

from langchain_core.runnables import RunnableLambda

def sanitize(text: str) -> str:
    return text.strip().lower()

def format_output(text: str) -> dict:
    return {"result": text, "length": len(text)}

chain = (
    RunnableLambda(sanitize)
    | llm
    | StrOutputParser()
    | RunnableLambda(format_output)
)

```

## 5.6 Chain of Thought Prompting

Guide the model to reason step by step:

```

template = ChatPromptTemplate.from_messages([
    ("system", "You are a math tutor. Always show
your reasoning step by step."),
    ("human", "Solve: {problem}"),
    ("ai", "Let me work through this step by
step:"),
])

chain = template | llm | StrOutputParser()

```

## 5.7 Error Handling

Wrap chain steps with error handling:

```
from langchain_core.runnables import RunnableWith-  
Fallbacks  
  
primary = prompt | llm | StrOutputParser()  
fallback = prompt_fallback | llm_backup | StrOut-  
putParser()  
  
chain = primary.with_fallbacks([fallback])
```

---

## Chapter 6: Memory – Giving Context to Conversations

### 6.1 Why Memory Matters

LLMs are stateless. Without memory, each call is isolated. Memory bridges this gap by storing and retrieving conversation history.

### 6.2 ConversationBufferMemory

The simplest memory keeps a buffer of recent messages:

```

from langchain.memory import ConversationBufferMemory
from langchain.chains import ConversationChain

memory = ConversationBufferMemory()
conversation = ConversationChain(llm=llm,
memory=memory)

response1 = conversation.predict(input="Hi, I am Alice.")
response2 = conversation.predict(input="What is my name?")
# The model remembers "Alice"

```

### 6.3 ConversationBufferWindowMemory

Limit memory to the last N exchanges to avoid context overflow:

```

from langchain.memory import
ConversationBufferWindowMemory

memory = ConversationBufferWindowMemory(k=5) #
last 5 exchanges

```

### 6.4 ConversationSummaryMemory

For long conversations, store a rolling summary instead of raw history:

```

from langchain.memory import ConversationSummaryMemory

memory = ConversationSummaryMemory(llm=llm)

```



The LLM periodically summarizes old messages, keeping only the compressed version.

## 6.5 Vector Store Memory

For very long-term memory, use a vector store to retrieve relevant past conversations:

```
from langchain.memory import VectorStoreRetriever-  
Memory  
from langchain_community.vectorstores import Chroma  
from langchain_openai import OpenAIEmbeddings  
  
vectorstore =  
Chroma(embedding_function=OpenAIEmbeddings())  
retriever = vector-  
store.as_retriever(search_kwargs={"k": 3})  
  
memory = VectorStoreRetriever-  
Memory(retriever=retriever)
```

## 6.6 Memory in LangChain Chains

When using LCEL, memory is added via [RunnableWithMessageHistory](#):

```

from langchain_core.runnables.history import RunnableWithMessageHistory
from langchain_core.chat_history import BaseChatMessageHistory

store = {}

def get_session_history(session_id: str) -> BaseChatMessageHistory:
    if session_id not in store:
        store[session_id] = InMemoryChatMessageHistory()
    return store[session_id]

chain = prompt | llm | StrOutputParser()

with_history = RunnableWithMessageHistory(
    chain,
    get_session_history,
    input_messages_key="input",
    history_messages_key="history",
)

result = with_history.invoke(
    {"input": "Hi, I am Alice."},
    config={"configurable": {"session_id": "user-123"}},
)

```



## 7.2 Document Loaders

LangChain supports hundreds of document loaders:

```
from langchain_community.document_loaders import
    TextLoader, PyPDFLoader, WebBaseLoader

# Load text file
docs = TextLoader("data.txt").load()

# Load PDF
docs = PyPDFLoader("report.pdf").load()

# Load web page
docs = WebBaseLoader("https://example.com").load()
```

## 7.3 Text Splitting

Documents must be split into chunks that fit within the LLM's context window:

```
from langchain.text_splitter import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=200,
    separators=["\n\n", "\n", ".", " ", ""],
)

chunks = splitter.split_documents(docs)
```

The overlap ensures that context is not lost at chunk boundaries.

## 7.4 Embeddings and Vector Stores

Convert chunks into vector embeddings and store them:

```
from langchain_openai import OpenAIEmbeddings
from langchain_community.vectorstores import Chroma

embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
vectorstore = Chroma.from_documents(chunks, embeddings)
```

## 7.5 Retrieval

Query the vector store for relevant documents:

```
retriever = vectorstore.as_retriever(
    search_type="similarity",
    search_kwargs={"k": 4},
)

results = retriever.invoke("What is machine learning?")
```

Advanced retrieval options:

```
# Maximum marginal relevance (diverse results)
retriever = vectorstore.as_retriever(
    search_type="mmr",
    search_kwargs={"k": 4, "fetch_k": 20,
"lambda_mult": 0.5},
)

# Similarity score threshold
retriever = vectorstore.as_retriever(
    search_type="similarity_score_threshold",
    search_kwargs={"score_threshold": 0.7},
)
```

## 7.6 Building the RAG Chain

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnablePassthrough

template = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful assistant. Use the context below to answer."),
    ("human", "Context:\n{context}\n\nQuestion: {question}"),
])

def format_docs(docs):
    return "\n\n".join(doc.page_content for doc in docs)

rag_chain = (
    {
        "context": retriever | format_docs,
        "question": RunnablePassthrough(),
    }
    | template
    | llm
    | StrOutputParser()
)

result = rag_chain.invoke("What is machine learning?")
```

## 7.7 Advanced RAG Techniques

**Hybrid Search** — Combine vector similarity with keyword search:

```
vectorstore = Chroma.from_documents(
    chunks,
    embeddings,
    collection_name="hybrid",
)
# Chroma supports both dense and sparse (BM25) re-
trieval
```

**Query Transformation** — Rewrite the user query before retrieval:

```
rewrite_template = ChatPromptTem-
plate.from_template(
    "Rewrite this question for better search:
    {question}"
)
rewriter = rewrite_template | llm | StrOutputPars-
er()

chain = (
    {"question": RunnablePassthrough()}
    | Runnable-
Passthrough.assign(search_query=rewriter)
    | (lambda x: {
        "context":
format_docs(retriever.invoke(x["search_query"])),
        "question": x["question"],
    })
    | template | llm | StrOutputParser()
)
```



**Multi-Query Retrieval** — Generate multiple search queries and aggregate results:

```
from langchain_core.runnables import RunnableParallel

queries_template = ChatPromptTemplate.from_template(
    "Generate 3 different search queries for: {question}"
)

queries_chain = queries_template | llm | StrOutputParser()

multi_retrieval = RunnableParallel(
    q1=lambda x: retriever.invoke(x["queries"].split("\n")[0]),
    q2=lambda x: retriever.invoke(x["queries"].split("\n")[1]),
    q3=lambda x: retriever.invoke(x["queries"].split("\n")[2]),
)
```

---

## Chapter 8: Tools and Toolkits

Tools are the bridge between LLMs and the external world. A tool is a function that the LLM can invoke, with a name, description, and parameter schema.

## 8.1 Defining a Tool

```
from langchain_core.tools import tool

@tool
def get_weather(city: str) -> str:
    """Get the current weather for a city."""
    # In production, call a weather API
    return f"The weather in {city} is sunny, 22°C."

@tool
def search_web(query: str) -> str:
    """Search the web for information."""
    # In production, use a search API
    return f"Search results for: {query}"
```

## 8.2 Binding Tools to a Model

Tool descriptions are sent to the LLM as part of the API call. The model can then request tool invocations:

```
tools = [get_weather, search_web]
llm_with_tools = llm.bind_tools(tools)

response = llm_with_tools.invoke("What is the
weather in Paris?")
# response may contain tool_calls
```

## 8.3 Executing Tool Calls

When the model requests a tool call, you must execute it and feed the result back:

```
from langchain_core.messages import AIMessage

response = llm_with_tools.invoke("What is the
weather in Paris?")

if response.tool_calls:
    for tool_call in response.tool_calls:
        tool_name = tool_call["name"]
        tool_args = tool_call["args"]
        # Execute the tool
        result = globals()[tool_name](**tool_args)
        print(f"{tool_name}({tool_args}) = {result}")
```

## 8.4 Using ToolNode (LangChain)

For a more structured approach, use LangChain's agent execution utilities:

```
from langchain.agents import create_tool_calling_agent, AgentExecutor
from langchain_core.prompts import ChatPromptTemplate

prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful assistant with access to tools."),
    ("human", "{input}"),
    ("placeholder", "{agent_scratchpad}"),
])

agent = create_tool_calling_agent(llm, tools, prompt)
agent_executor = AgentExecutor(agent=agent, tools=tools, verbose=True)

result = agent_executor.invoke({"input": "What is the weather in Tokyo?"})
```

## 8.5 Built-in Toolkits

LangChain provides many pre-built toolkits:

```

from langchain_community.agent_toolkits import (
    SQLDatabaseToolkit,
    GitHubToolkit,
    FileManagementToolkit,
)

# SQL database toolkit
from langchain_community.utilities import SQLDatabase
db = SQLDatabase.from_uri("sqlite:///mydb.sqlite")
sql_toolkit = SQLDatabaseToolkit(db=db, llm=llm)

# File management toolkit
file_toolkit = FileManagementToolkit(root_dir="./data")

```

## 8.6 Custom Tool Design

Good tools have:

1. **Descriptive names** — `get_weather`, not `func_a`
2. **Clear descriptions** — Explain when and why the tool should be used
3. **Well-typed parameters** — Use Python type hints with descriptions
4. **Deterministic behavior** — Same input always produces same output
5. **Error handling** — Catch exceptions and return meaningful error messages

```

from typing import Annotated
from langchain_core.tools import tool

@tool
def get_stock_price(
    symbol: Annotated[str, "Stock ticker symbol,
e.g., AAPL"],
    date: Annotated[str, "Date in YYYY-MM-DD
format"],
) -> str:
    """Get the closing stock price for a given sym-
bol and date."""
    try:
        # Call financial API
        return f"{symbol} closed at $150.25 on
{date}."
    except Exception as e:
        return f"Error fetching stock price: {e}"

```

## 8.7 Tool Security Considerations

- **Validate inputs** — Never trust tool arguments from LLMs.
- **Limit scope** — Restrict file system, network, and database access.
- **Audit logging** — Log all tool invocations with inputs and outputs.
- **Rate limiting** — Protect downstream APIs from excessive calls.
- **User confirmation** — Require approval for destructive operations.

# Part III: Agents with LangGraph

---

## Chapter 9: From Linear Chains to Graphs

### 9.1 The Limitations of Chains

Chains are linear: step A  $\rightarrow$  step B  $\rightarrow$  step C. Real-world agents need more:

- **Loops** — An agent may need to call a tool, analyze the result, and call another tool.
- **Branching** — Different paths based on intermediate results.
- **State** — Carry context across multiple steps.
- **Persistence** — Survive failures and restarts.

LangGraph addresses all of these.

### 9.2 Graph Concepts

A LangGraph application is a directed graph with:

- **State** — A shared data structure that persists across all nodes.
- **Nodes** — Python functions that read and write state.
- **Edges** — Connections that define the flow between nodes.

- **Conditional Edges** — Functions that determine which node to go to next based on state.

## 9.3 Your First LangGraph Application

```
from typing import TypedDict, List
from langgraph.graph import StateGraph, END
from langchain_core.messages import HumanMessage,
AIMessage

class AgentState(TypedDict):
    messages: List[HumanMessage | AIMessage]

def chatbot(state: AgentState) -> AgentState:
    last_message = state["messages"][-1]
    response = llm.invoke(state["messages"])
    return {"messages": [response]}

# Build the graph
graph = StateGraph(AgentState)

# Add nodes
graph.add_node("chatbot", chatbot)

# Set entry and exit points
graph.set_entry_point("chatbot")
graph.add_edge("chatbot", END)

# Compile
app = graph.compile()

# Run
result = app.invoke({
    "messages": [HumanMessage(content="Hello!")]
})
print(result["messages"][-1].content)
```



## 9.4 The Compilation Step

`graph.compile()` transforms the logical graph into an executable. During compilation, `LangGraph`:

1. Validates the graph structure
  2. Resolves edge conditions
  3. Optionally adds checkpointing and persistence
  4. Returns a `CompiledStateGraph` with `invoke`, `stream`, and `astream` methods
- 

# Chapter 10: StateGraph – The Heart of LangGraph

## 10.1 Defining State

State is the data that flows through your graph. It is defined as a Python `TypedDict` or a Pydantic `BaseModel`:

```
from typing import TypedDict, Annotated, List
from langgraph.graph import StateGraph
from langchain_core.messages import BaseMessage
import operator

class AgentState(TypedDict):
    messages: Annotated[List[BaseMessage], operator.add]
    next_agent: str
    documents: List[str]
    user_info: dict
```

Using `Annotated` with `operator.add` tells LangGraph to merge lists rather than overwrite them.

## 10.2 State Reducers

Reducers control how state updates are applied. Without a reducer, new values overwrite old ones:

```
class SimpleState(TypedDict):  
    counter: int # overwritten each time
```

With a reducer, values are combined:

```
from typing import Annotated  
import operator  
  
class AccumulatingState(TypedDict):  
    steps: Annotated[List[str], operator.add]  
    total: Annotated[int, operator.add]
```

Custom reducers are also possible:

```
def my_reducer(existing: str | None, new: str) ->  
    str:  
    if existing is None:  
        return new  
    return f"{existing}\n{new}"  
  
class CustomState(TypedDict):  
    log: Annotated[str, my_reducer]
```

## 10.3 Adding Nodes

Nodes are functions that take state and return a state update:

```
def node_a(state: AgentState) -> dict:
    return {"messages": [AIMessage(content="Hello
from A")]}

def node_b(state: AgentState) -> dict:
    return {"messages": [AIMessage(content="Hello
from B")]}

graph = StateGraph(AgentState)
graph.add_node("node_a", node_a)
graph.add_node("node_b", node_b)
```

## 10.4 Setting Entry Point

Every graph needs exactly one entry point:

```
graph.set_entry_point("node_a")
```

## 10.5 Adding Edges

Simple edges connect one node to another:

```
graph.add_edge("node_a", "node_b")
graph.add_edge("node_b", END)
```

`END` is a special constant that terminates execution.

## 10.6 Compilation and Execution

```
app = graph.compile()
result = app.invoke({"messages": []})
```

## 10.7 Streaming Graph Execution

LangGraph supports streaming at the level of state updates:

```
for event in app.stream({"messages": []}):
    for node_name, state_update in event.items():
        print(f"[{node_name}] {state_update}")
```

---

# Chapter 11: Nodes, Edges, and Conditional Routing

## 11.1 Conditional Edges

Conditional edges route to different nodes based on state:

```
def router(state: AgentState) -> str:
    last = state["messages"][-1]
    if "tool_call" in last:
        return "tools"
    return "response"

graph.add_conditional_edges(
    "agent",
    router,
    {
        "tools": "execute_tools",
        "response": "format_response",
    }
)
```

The router function receives the current state and returns a key. The mapping dictionary translates keys to node names.

## 11.2 Branching Logic

More complex branching with multiple conditions:

```

def classify_input(state: AgentState) -> str:
    question = state["messages"][-1].content.lower()
    if "weather" in question:
        return "weather_agent"
    elif "news" in question:
        return "news_agent"
    elif "calculate" in question:
        return "calculator_agent"
    else:
        return "general_agent"

graph.add_conditional_edges(
    "classifier",
    classify_input,
    {
        "weather_agent": "weather_node",
        "news_agent": "news_node",
        "calculator_agent": "calculator_node",
        "general_agent": "general_node",
    }
)

```

## 11.3 Loops and Cycles

Unlike chains, graphs support cycles. This is essential for agent loops:

```

# Agent loop: agent -> tools -> agent -> ... -> END
graph.add_edge("agent", "tools")
graph.add_conditional_edges(
    "tools",
    should_continue,
    {"continue": "agent", "end": END},
)

```

The `should_continue` function might check if the agent's latest response contains tool calls:

```
def should_continue(state: AgentState) -> str:
    last = state["messages"][-1]
    if hasattr(last, "tool_calls") and last.tool_calls:
        return "continue"
    return "end"
```

## 11.4 Parallel Execution

Nodes can run in parallel using fan-out edges:

```
graph.add_edge("analyzer", "summarizer")
graph.add_edge("analyzer", "fact_checker")
graph.add_edge("analyzer", "sentiment_analyzer")

# All three run in parallel after analyzer completes
graph.add_edge("summarizer", "aggregator")
graph.add_edge("fact_checker", "aggregator")
graph.add_edge("sentiment_analyzer", "aggregator")
```

## 11.5 Subgraphs

For complex workflows, compose graphs within graphs:

```
research_graph = StateGraph(ResearchState)
# ... define research nodes and edges ...

main_graph = StateGraph(MainState)
main_graph.add_node("research", re-
search_graph.compile())
main_graph.add_node("write", write_node)
main_graph.add_edge("research", "write")
main_graph.add_edge("write", END)
```

---

## Chapter 12: Agent Loops and Reasoning

### 12.1 The ReAct Pattern

The most common agent architecture is ReAct (Reasoning + Acting). The agent:

1. Thinks about what to do next
2. Calls a tool if needed
3. Observes the result
4. Repeats until it can answer



## 12.2 Building a ReAct Agent in LangGraph

```

from typing import TypedDict, List, Annotated
from langgraph.graph import StateGraph, END
from langgraph.prebuilt import ToolNode
from langchain_core.messages import BaseMessage,
HumanMessage, AIMessage, FunctionMessage
import operator

class AgentState(TypedDict):
    messages: Annotated[List[BaseMessage], operator.add]

tools = [get_weather, search_web]
tool_node = ToolNode(tools)

model = llm.bind_tools(tools)

def call_model(state: AgentState) -> dict:
    result = model.invoke(state["messages"])
    return {"messages": [result]}

def should_continue(state: AgentState) -> str:
    last = state["messages"][-1]
    if hasattr(last, "tool_calls") and last.tool_calls:
        return "continue"
    return "end"

# Build graph
graph = StateGraph(AgentState)
graph.add_node("agent", call_model)
graph.add_node("tools", tool_node)

graph.set_entry_point("agent")
graph.add_conditional_edges(
    "agent",
    should_continue,
    {"continue": "tools", "end": END},
)

```

```
graph.add_edge("tools", "agent")

app = graph.compile()

# Run
result = app.invoke({
    "messages": [HumanMessage(content="What is the
weather in Paris and Tokyo?")]
})
```

## 12.3 Limiting Agent Loops

Prevent infinite loops with maximum iterations:

```
from langgraph.graph import StateGraph

# Add recursion limit
app = graph.compile()
result = app.invoke(
    {"messages": [HumanMessage(content="Search for
AI news.")]},
    {"recursion_limit": 25}, # max node executions
)
```

## 12.4 Structured Output with Pydantic

Guide the agent's reasoning with structured outputs:

```

from pydantic import BaseModel, Field

class AgentThought(BaseModel):
    reasoning: str = Field(description="Step-by-step reasoning")
    action: str = Field(description="Next action: 'tool' or 'respond'")
    tool_name: str | None = Field(description="Tool to call, if action is 'tool'")
    tool_args: dict | None = Field(description="Arguments for the tool")
    final_answer: str | None = Field(description="Final answer, if action is 'respond'")

structured_llm = llm.with_structured_output(AgentThought)

```

## 12.5 Multi-Step Reasoning

For complex tasks, force the agent to decompose:

```

def decompose(state: AgentState) -> dict:
    task = state["messages"][-1].content
    prompt = f"""
    Break this task into subtasks:
    {task}

    List each subtask on a new line, numbered.
    """
    result = llm.invoke(prompt)
    return {"subtasks": result.content.split("\n")}

def execute_subtask(state: AgentState) -> dict:
    current = state["subtasks"]
    [state["current_index"]]
    # Execute the subtask with tools
    ...
    return {"results": [...], "current_index":
state["current_index"] + 1}

```

## Chapter 13: Human-in-the-Loop Patterns

### 13.1 Why Human-in-the-Loop?

For high-stakes applications (financial transactions, medical decisions, content moderation), letting an agent act autonomously is risky. Human-in-the-loop (HITL) pauses execution at key points for human approval.

## 13.2 Using Interrupts

LangGraph provides the `interrupt` function to pause graph execution:

```
from langgraph.types import interrupt

def approve_action(state: AgentState) -> dict:
    last = state["messages"][-1]
    if not hasattr(last, "tool_calls") or not
last.tool_calls:
        return {}

    tool_call = last.tool_calls[0]

    decision = interrupt({
        "question": "Approve this tool call?",
        "tool": tool_call["name"],
        "args": tool_call["args"],
    })

    if decision == "approve":
        return {"approved": True}
    else:
        return {"approved": False, "messages":
[AIMessage(content="Action rejected by user.")]}
```

## 13.3 Resuming Execution

When execution is interrupted, it can be resumed with human input:

```
# Initial run
initial_result = app.invoke({
    "messages": [HumanMessage(content="Transfer
$1000 to account 12345.")]
})

# The graph pauses at approve_action
# Resume with user's decision
resumed_result = app.invoke(
    None,
    config={"configurable": {"thread_id":
"thread-1"}},
    # Command.resume provides the human decision
    # (LangGraph handles this through the
    interrupt/resume API)
)
```

## 13.4 Thread-Level Persistence

For HITL to work, the graph must persist its state between invocations. This requires a checkpointer:

```
from langgraph.checkpoint.memory import MemorySaver

checkpointer = MemorySaver()
app = graph.compile(checkpointer=checkpointer)
```

Each thread has a unique `thread_id` that identifies the conversation.

## 13.5 HITL Patterns

**Approval Before Tool Execution** — Pause before calling any tool:

```
agent -> [interrupt: approve?] -> tools -> agent  
-> ...
```

**Approval After Analysis** — Let the agent analyze, then ask for approval:

```
agent -> analyze -> [interrupt: proceed?] -> agent  
-> ...
```

**Escalation** — Route to a human agent when confidence is low:

```
def router(state: AgentState) -> str:  
    if state["confidence"] < 0.7:  
        return "human_escalation"  
    return "auto_response"
```

---

## Chapter 14: Persistence and Long-Term Memory

### 14.1 Checkpointing

LangGraph's durable execution means the graph state is saved after each node execution. If the process crashes, it resumes from the last checkpoint.



```

from langgraph.checkpoint.memory import MemorySaver
from langgraph.checkpoint.sqlite import SqliteSaver

# In-memory (development)
checkpointer = MemorySaver()

# SQLite (production – single process)
checkpointer = Sql-
iteSaver.from_conn_string("checkpoints.sqlite")

# PostgreSQL (production – distributed)
# Use LangGraph's Postgres checkpointer

```

## 14.2 Enabling Persistence

```

app = graph.compile(checkpointer=checkpointer)

# Each session gets a unique thread_id
config = {"configurable": {"thread_id": "user-ses-
sion-42"}}

result = app.invoke(
    {"messages": [HumanMessage(content="Hi, I am
Alice.")]},
    config=config,
)

result = app.invoke(
    {"messages": [HumanMessage(content="What is my
name?")]},
    config=config, # Same thread_id = remembered
context
)

```

## 14.3 Memory Across Sessions

For long-term memory that persists across threads, use LangGraph's store:

```
from langgraph.store.base import BaseStore
from langgraph.store.memory import InMemoryStore

store = InMemoryStore()

# Save user preferences
store.put(
    ("users", "alice"),
    "preferences",
    {"language": "Spanish", "tone": "formal"},
)

# Retrieve in any session
prefs = store.get(("users", "alice"), "preferences")
```

## 14.4 Working with Checkpoints

List and inspect checkpoints:

```
# Get all checkpoints for a thread
checkpoints = app.list(config)

# Get specific checkpoint
checkpoint = app.get_state(config)

# Replay from a checkpoint
for event in app.istream_events(
    None,
    config,
    version="v2",
):
    ...
```

## 14.5 State Management Best Practices

1. **Keep state minimal** — Store only what the graph needs.
  2. **Use reducers wisely** — `operator.add` for lists, custom reducers for specific merge logic.
  3. **Version your state schema** — Plan for evolution as your application grows.
  4. **Clean up old threads** — Set TTL on checkpoints to manage storage.
  5. **Separate ephemeral from persistent** — Use state for short-term, store for long-term.
-

## Part IV: Production and Beyond

---

### Chapter 15: Observability with LangSmith

#### 15.1 Why Observability Matters

LLM applications are inherently non-deterministic. You need visibility into:

- What prompt was sent to the model
- What the model responded
- Which tools were called and with what arguments
- How long each step took
- Where errors occurred

LangSmith provides this observability.

#### 15.2 Setting Up LangSmith

```
import os
os.environ["LANGSMITH_TRACING"] = "true"
os.environ["LANGSMITH_API_KEY"] = "lsv2_..."
os.environ["LANGSMITH_PROJECT"] = "my-agent"
```

Once set, all LangChain calls are automatically traced.

## 15.3 Tracing

Every `invoke`, `stream`, and `batch` call is recorded as a trace:

- **Run trees** — Hierarchical view of calls (chain → LLM → tool)
- **Timing** — Duration of each step
- **Token counts** — Input and output tokens per LLM call
- **Error tracking** — Stack traces and failed calls

## 15.4 Custom Traces

Add rich metadata to traces:

```
from langchain.callbacks.tracers import LangSmithTracer

tracer = LangSmithTracer()
config = {
    "tags": ["production", "user-request"],
    "metadata": {
        "user_id": "123",
        "session_id": "abc",
    },
}

result = chain.invoke({"input": "Hello"}, config=config)
```

## 15.5 Evaluation

LangSmith supports dataset-based evaluation:

```

from langsmith import Client

client = Client()

# Create a dataset
dataset = client.create_dataset("qa-evaluation")
client.create_examples(
    dataset_id=dataset.id,
    inputs=[{"question": "What is 2+2?"}],
    outputs=[{"answer": "4"}],
)

# Run evaluation
results = client.evaluate(
    lambda inputs:
chain.invoke(inputs["question"]),
    data=dataset.id,
    evaluators=[...],
)

```

## 15.6 Debugging with LangSmith

When an agent behaves unexpectedly:

1. Find the trace in LangSmith
  2. Examine the exact prompts sent to each LLM call
  3. Check tool call arguments and results
  4. Look at timing bottlenecks
  5. Review error messages
-

## Chapter 16: Deploying Agentic Applications

### 16.1 LangGraph Server

The LangGraph Server turns your compiled graph into a REST API:

```
pip install "langgraph-cli[inmem]"
langgraph dev
```

This starts a local server with:

- `POST /runs/` — Invoke the graph
- `GET /runs/{run_id}` — Get run status
- `POST /runs/{run_id}/stream` — Stream results
- Thread management endpoints

### 16.2 LangGraph Platform

For production, deploy to the LangGraph Platform (managed) or self-host:

```
# langgraph.json
{
  "name": "my-agent",
  "version": "1.0.0",
  "graphs": {
    "agent": "./src/agent.py:graph"
  },
  "env": {
    "OPENAI_API_KEY": "${OPENAI_API_KEY}"
  }
}
```

## 16.3 Self-Hosted Deployment

Deploy as a FastAPI application:

```
from fastapi import FastAPI
from langgraph.graph import StateGraph

app = FastAPI()
graph = build_graph()
compiled = graph.compile(checkpointer=checkpointer)

@app.post("/invoke")
async def invoke(messages: list, thread_id: str):
    config = {"configurable": {"thread_id":
thread_id}}
    result = await compiled.ainvoke(
        {"messages": messages},
        config=config,
    )
    return {"response": result["messages"][-1].content}
```



## 16.4 Scaling Considerations

- **Stateless vs. stateful** — Stateless chains scale horizontally easily. Stateful graphs need a shared checkpointer (PostgreSQL, Redis).
  - **Rate limiting** — Protect LLM APIs with rate limiters.
  - **Caching** — Cache LLM responses for identical inputs.
  - **Batching** — Use `batch` for non-interactive workloads.
  - **Monitoring** — Set up alerts for latency, error rates, and token usage.
- 

## Chapter 17: Multi-Agent Systems

### 17.1 Why Multiple Agents?

Complex tasks often require specialized agents:

- A research agent gathers information
- A writer agent produces content
- A reviewer agent checks quality
- A coordinator agent manages the workflow

### 17.2 Agent Communication Patterns

**Sequential** — One agent after another:

```
graph.add_edge("researcher", "writer")
graph.add_edge("writer", "reviewer")
graph.add_edge("reviewer", END)
```

**Supervisor** — A managing agent delegates to specialists:

```
def supervisor(state: AgentState) -> str:
    task = state["current_task"]
    if task["type"] == "research":
        return "researcher"
    elif task["type"] == "code":
        return "coder"
    return "general"

graph.add_conditional_edges("supervisor",
supervisor, {...})
```

**Debate** — Multiple agents discuss and converge:

```
def debate_round(state: AgentState) -> dict:
    positions = []
    for agent in state["agents"]:
        position =
agent.invoke(state["debate_context"])
        positions.append(position)
    return {"positions": positions}
```

## 17.3 Building a Supervisor Agent

```
class SupervisorState(TypedDict):
    messages: Annotated[List[BaseMessage], operator.add]
    current_agent: str
    task_completed: bool

supervisor_prompt = ChatPromptTemplate.from_messages([
    ("system", "You supervise a team of agents: researcher, writer, reviewer."),
    ("human", "Task: {task}\nCurrent progress: {progress}\nWhich agent should work next?"),
])

def supervisor_node(state: SupervisorState) -> dict:
    prompt = supervisor_prompt.invoke({
        "task": state["task"],
        "progress": state["messages"][-1].content,
    })
    decision = llm.invoke(prompt)
    return {"current_agent": decision.content.strip()}
```

## 17.4 Shared State Across Agents

All agents in the graph share the same state. Design your state schema to give each agent the data it needs:

```
class MultiAgentState(TypedDict):
    messages: Annotated[List[BaseMessage], operator.add]
    research_notes: List[str]
    draft: str
    review_comments: List[str]
    approved: bool
```

## 17.5 Coordination Challenges

1. **Context overflow** — The shared state can grow large. Use summarization or sliding windows.
  2. **Conflicting outputs** — Define resolution rules when agents disagree.
  3. **Deadlocks** — Ensure the graph always reaches END.
  4. **Latency** — Parallel execution where possible; set timeouts on agent nodes.
- 

# Chapter 18: From Prototype to Production

## 18.1 The Development Lifecycle

1. **Prototype** — Jupyter notebook, one script, quick iteration
2. **Test** — Unit tests for nodes, integration tests for the full graph
3. **Evaluate** — Dataset evaluation with LangSmith
4. **Deploy** — LangGraph Server or custom API
5. **Monitor** — Traces, metrics, and alerts

## 6. Iterate — Improve prompts, add tools, refine logic

### 18.2 Testing Strategies

```
# Test individual nodes
def test_call_model():
    state = {"messages": [HumanMessage(content="Say hello")]}
    result = call_model(state)
    assert len(result["messages"]) == 1
    assert isinstance(result["messages"][0], AIMessage)

# Test full graph
def test_full_graph():
    app = graph.compile()
    result = app.invoke({"messages": [HumanMessage(content="Hello")]}))
    assert result["messages"][-1].content

# Test with mocked LLM
class MockLLM:
    def invoke(self, messages):
        return AIMessage(content="Mock response")

def test_with_mock(monkeypatch):
    monkeypatch.setattr("my_agent.llm", MockLLM())
    result = my_node({"messages": []})
    assert result["messages"][0].content == "Mock response"
```

### 18.3 Performance Optimization

- **Cache embeddings** — Embedding calls are expensive. Cache them.

- **Parallelize independent operations** — Use `RunnableParallel` and graph fan-out.
- **Stream responses** — Show partial results to users instead of blocking.
- **Batch processing** — Process multiple inputs in one pass.
- **Model selection** — Use cheaper models for simple tasks, larger models for complex reasoning.

## 18.4 Security Checklist

- [ ] API keys stored in environment variables or secrets manager
- [ ] Tool inputs validated and sanitized
- [ ] Rate limiting on all external API calls
- [ ] Logging of all LLM interactions
- [ ] User authentication and authorization
- [ ] Data isolation between users/tenants
- [ ] Prompt injection mitigations
- [ ] Regular dependency updates

## 18.5 Common Pitfalls

| Pitfall                       | Solution                                               |
|-------------------------------|--------------------------------------------------------|
| Agent gets stuck in loop      | Set <code>recursion_limit</code> and add timeout nodes |
| Context window overflow       | Compress history with summary or window                |
| LLM hallucinates data         | Ground with RAG, add fact-checking tools               |
| Tool failures crash the agent | Wrap tools in try/except, return error messages        |
| Prompts degrade over time     | Version prompts, run eval suite regularly              |

## 18.6 What's Next?

The ecosystem evolves rapidly. Stay current by:

- Following the [LangChain blog](#)
- Monitoring the [LangGraph changelog](#)
- Joining the [LangChain community forum](#)
- Building and iterating on real projects

The most important thing you can do is build. Start with a simple chatbot, add RAG, give it tools, and gradually increase autonomy. Each step teaches you what works and what doesn't in your specific domain.

# Appendices

---

## Appendix A: Environment Setup

```
# Create and activate virtual environment
python -m venv .venv
source .venv/bin/activate # On Windows: .venv\Scripts\activate

# Install core packages
pip install langchain langchain-openai langchain-anthropic langchain-community
pip install langgraph langgraph-checkpoint
pip install chromadb tiktoken pypdf python-dotenv
pip install "langgraph-cli[inmem]"
pip install jupyter # for notebooks

# .env file
echo "OPENAI_API_KEY=sk-..." > .env
echo "LANGSMITH_API_KEY=lsv2_..." >> .env
```



## Appendix B: Python Virtual Environments

```
# macOS/Linux
python -m venv .venv
source .venv/bin/activate

# Windows
python -m venv .venv
.venv\Scripts\activate

# Deactivate
deactivate

# Export dependencies
pip freeze > requirements.txt

# Install from requirements
pip install -r requirements.txt
```

## Appendix C: Common Troubleshooting

### Module not found

```
pip install <missing-package>
```

### API key errors

Check that your `.env` file exists and keys are correct.  
Ensure `load_dotenv()` is called before any `LangChain` imports.

## Token limit exceeded

Reduce `chunk_size` in text splitting.  
Use summary memory instead of buffer memory.  
Switch to a model with larger context window.

## Graph execution stuck

Check for missing edges or unreachable nodes.  
Verify the router function returns valid keys.  
Increase `recursion_limit` cautiously.

## Tool call failures

Wrap tool implementations in `try/except`.  
Test tools in isolation before adding to the graph.  
Check tool parameter schemas match the model's output.

## Appendix D: Further Resources

- [LangChain Documentation](#)
- [LangGraph Documentation](#)
- [LangSmith Documentation](#)
- [LangChain Academy \(Free Courses\)](#)
- [LangChain GitHub](#)
- [LangGraph GitHub](#)
- [LangChain Forum](#)
- [Python LangGraph Reference](#)

---

## About the Author

**Kairat Yussupov** is a software engineer and technical author with deep expertise in building AI-powered applications. His work focuses on agentic AI, LLM orchestration, and production-grade machine learning systems. He writes about the intersection of software engineering and artificial intelligence, helping developers navigate the rapidly evolving AI ecosystem.

\newpage

\*Thank you for reading. Now go build something autonomous.\*